

PHASEMATRIX-HIVEMIND-03

Morphodynamisches Resonanzzell-Substrat

Formale Implementierungsspezifikation

Sebastian Klemm

Mai 2026

Zusammenfassung

Diese Spezifikation definiert ein eigenständiges, implementierungsbereites PhaseMatrix-Profil für ein morphodynamisches Resonanzzell-Substrat. Der Entwurf beschreibt eine föderierte Phasenraum-Matrix aus Nodes, Subnetzen, Claims, Boundary-Gates, Truth-Maintenance, Zellpools, resonanzgekoppelten Clustern, FunnelGraphs, Konvergenzfeldern, morphodynamischen Ereignissen, IntentCandidates, trace-erhaltender Working-State-Kompaktion und optionalem Handoff an nachgelagerte Handoff-Schichten. Das Dokument ist vollständig aus sich heraus zu lesen: Alle implementierungsrelevanten Typen, Operatoren, Gates, Reports, CLI-Kommandos, Tests und Definition-of-Done-Regeln sind enthalten. Die Spezifikation beschreibt ausschließlich domänenneutrale Laufzeit-, Daten- und Audit-Mechanik.

Inhaltsverzeichnis

1	Status, Geltungsbereich und Zweck	5
1.1	Status	5
1.2	Zweck	5
1.3	Geltungsbereich	5
1.4	Nicht-Ziele	5
2	Begriffskern und technische Primitive	5
2.1	Implementierungsbegriffe	5
2.2	Domänenneutralität	6
3	Architekturposition	6
3.1	Gesamtstack	6
3.2	Schichtrollen	7
4	Mathematischer Kern	7
4.1	TridentVector	7
4.2	MorphodynamicField	7
4.3	ConvergenceField	8
5	Kanonische Primitive	8
5.1	Zahl- und Hashregeln	8
5.2	Basistypen	8
6	Matrix-Basistypen	8
6.1	PhaseMatrixNode und PhaseSubnet	8
6.2	Claims, Truth-Maintenance und Boundary	9
6.3	MatrixTrace und Gate-Policy	10
7	Run Descriptor v0.3	10
8	Datenmodelle: PhaseCell-Schicht	11
8.1	TridentVector	11
8.2	PhaseCell	11
8.3	CellPool	12
9	LocalResonanceProcessoren	12
9.1	LocalResonanceProcessor	12
9.2	ResonancePulse	12
10	MorphodynamicField	13
10.1	Datenstruktur	13
10.2	MorphologyEvent	13
11	ResonanceCluster	14
11.1	Cluster-Modell	14
11.2	ClusterFormationReport	14
12	FunnelGraph	15

12.1	Zweck	15
12.2	Datenstruktur	15
13	ConvergenceField	16
13.1	Datenstruktur	16
13.2	Konvergenzregel	16
14	TensionToIntentOperator	16
14.1	Zweck	16
14.2	Datenmodell	16
15	RecursiveFeedback	17
15.1	Bounded Feedback	17
15.2	Feedback-Regel	17
16	Dissolution und Trace-Erhaltung	17
16.1	Grundsatz	17
16.2	Datenmodell	18
17	ClusterTrace	18
18	Gates	18
18.1	Cluster Formation Gate	18
18.2	Morphology Gate	18
18.3	Intent Gate	18
18.4	Dissolution Gate	19
18.5	Matrix Boundary Gate	19
19	Pipeline v0.3	19
19.1	Referenzablauf	19
19.2	Pseudocode	19
20	Integration mit PhaseMatrix und optionalen Handoff-Schichten	20
20.1	Matrix-Integration	20
20.2	Handoff	20
21	Outcome-Typen	21
22	CLI-Schnittstelle	21
22.1	Subcommands	21
22.2	CLI-Regeln	21
23	Feature Flags und Modulstruktur	22
23.1	Crate-Struktur	22
23.2	Feature Flags	22
24	Tests	23
24.1	Unit Tests	23
24.2	Integration Tests	23
24.3	Negative Tests	23

25 Definition of Done	23
26 Coding-Agent-Auftrag	24
27 Schlussformel	25

1 Status, Geltungsbereich und Zweck

1.1 Status

Dieses Dokument ist eine normative Implementierungsspezifikation. Die Begriffe **MUST**, **MUST NOT**, **SHOULD** und **MAY** sind verbindlich für Konformität.

1.2 Zweck

Ziel ist eine eigenständige Mikro- und Mesostruktur für PhaseMatrix-Subnetze. Ein Subnetz soll nicht nur aus statischen Nodes bestehen, sondern aus resonanzfähigen PhaseCells, die sich zu temporären oder persistenten Arbeitsclustern bilden, ihre lokale Morphologie kontrolliert anpassen, Arbeitszustände kompaktieren, Claims erzeugen und dabei Trace, Evidence, Governance und Matrix-Gates erhalten.

Kernthese

Ein PhaseSubnet ist kein statisches Graphobjekt. Es ist ein kontrolliertes morphodynamisches Feld aus PhaseCells, ResonanceClusters, FunnelGraphs, ConvergenceFields und Lifecycle-Gates. Jede emergente Zellstruktur darf lokal arbeiten und sich wieder auflösen; ihre Gate-Entscheidungen, Evidence und Trace-Artefakte bleiben dauerhaft auditierbar.

1.3 Geltungsbereich

Die Spezifikation definiert:

- die minimalen Matrix-Typen, die fuer Zellcluster, Claims, Boundary-Gates und Handoffs erforderlich sind;
- eine morphodynamische Zellschicht aus PhaseCells, CellPools und ResonanceClusters;
- FunnelGraphs für clusterinterne Traversierung ueber räumliche, zeitliche, semantische und resonante Kantenklassen;
- ConvergenceFields und TensionToIntent zur Erzeugung von IntentCandidates;
- trace-erhaltende Dissolution als Working-State-Kompaktion;
- Fail-Closed-Gates, Reports, Replay, CLI, Tests und Coding-Agent-Auftrag.

1.4 Nicht-Ziele

Die Spezifikation definiert keine fachliche Domänenanwendung, kein Kryptographieprotokoll, keinen Netzwerktransport, keine physische Aktorik und keinen wissenschaftlichen Wirklichkeitsclaim. Der Core ist ausschließlich ein deterministisches, auditierbares Daten-, Gate- und Laufzeitmodell. Fachliche Adapter **MAY** außerhalb des Core angebunden werden, **MUST** aber dessen Gates, Tracepflicht und Boundary-Regeln respektieren.

2 Begriffskern und technische Primitive

2.1 Implementierungsbegriffe

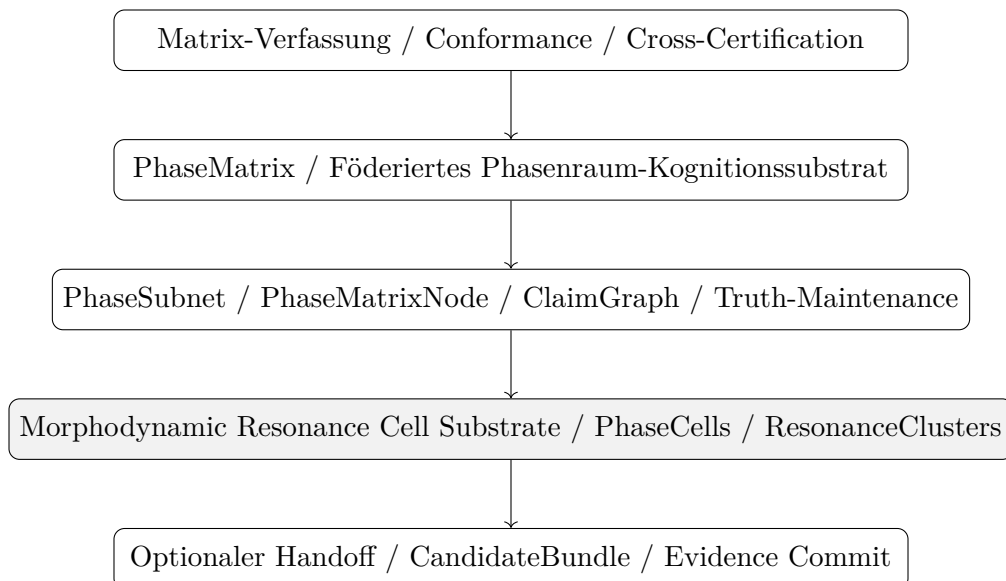
Begriff	Implementierungsbedeutung
PhaseCell	Lokaler, hashbarer Resonanzzustand mit TridentVector, Phase, Rolle, Lifecycle und Evidence.
LocalResonanceProcessor	Deterministische Eingabeprojektion von Inputdaten auf ResonancePulse und TridentVector.
ResonanceCluster	Temporäre oder persistente Arbeitsgruppe aus PhaseCells, gebunden an Purpose, Trace und MatrixBoundary.
FunnelGraph	Clusterinterne Traversalstruktur mit räumlichen, zeitlichen, semantischen und resonanten Kantenklassen.
ConvergenceField	Überlappungs- und Stabilisierungsschicht fuer Zellpulse, Konfliktwerte und Intent-Vorstufen.
MorphodynamicField	Growth-/Decay-/Split-/Fuse-Steuerfeld aus PhaseResonance und GrowthPotential.
TensionToIntentOperator	Gate-gebundene Transformation von Feldspannung und Konvergenz in IntentCandidates.
Dissolution	Working-State-Kompaktion: flüchtiger Arbeitszustand darf entfernt werden, Trace/Evidence/Gate-History bleiben erhalten.

2.2 Domänenneutralität

Alle public APIs, CLI-Kommandos, Reports und Tests **MUST** die hier definierten technischen Begriffe verwenden. Domänenspezifische Adapter **MAY** eigene Begriffe oberhalb des Core verwenden, **MUST** aber vor Eintritt in den Core auf die neutralen Typen dieser Spezifikation abbilden.

3 Architekturposition

3.1 Gesamtstack



3.2 Schichtrollen

Schicht	Rolle
Matrix-Verfassung	Konstitutionelle Regeln, Tracepflicht, Change-Sets, Proof-Obligations, Truth-Maintenance.
PhaseSubnet	Föderierte, lokal souveräne Teilmatrix mit Claims, Seams, Lattice und Governance.
CellPool	Vorrat an PhaseCells eines Nodes oder Subnetzes.
ResonanceCluster	Temporäre oder persistente Arbeitsgruppe aus PhaseCells.
FunnelGraph	Interne Cluster-Traversalstruktur ueber Raum/Zeit/Semantik/Resonanz.
ConvergenceField	Stabilisiert Zellpulse und erzeugt IntentCandidate-Vorstufen.
Dissolution	Kompaktiert Working State und erzeugt persistente Trace-Artefakte.

4 Mathematischer Kern

4.1 TridentVector

Jede PhaseCell und jeder ResonanceCluster besitzt einen normierten TridentVector:

$$T(t) = (s(t), c(t), p(t))$$

mit:

- $s(t)$: semantic density / Bedeutungsdichte,
- $c(t)$: structural coherence / strukturelle Kohärenz,
- $p(t)$: temporal phase readiness / zeitliche Phasenbereitschaft.

Das Aktivierungspotential lautet:

$$D(t) = s(t) \cdot c(t) \cdot p(t).$$

Ein lokaler Trigger ist nur zulaessig, wenn:

$$D(t) \cdot L_T(t) > \theta_D$$

und alle Trace-, Gate- und Boundary-Bedingungen bestanden sind. $L_T(t)$ ist eine deterministische temporale Envelope- oder Horizon-Funktion.

4.2 MorphodynamicField

Die morphodynamische Wachstumsschicht ist:

$$H(x, t) = \alpha \cdot \Phi(x, t) + \beta \cdot \mu(x, t)$$

mit:

- $\Phi(x, t)$: phase resonance field,
- $\mu(x, t)$: morphodynamic growth field,
- α, β : run-descriptor-gebundene Gewichtungen.

H steuert nicht direkt Output. Es steuert nur Morphologieentscheidungen wie Grow, Split, Fuse, Decay, Stabilize oder WorkingStateCompaction.

4.3 ConvergenceField

Ein ConvergenceField akkumuliert Zellpulse:

$$C_{field}(t+1) = \text{Normalize} \left(\sum_i P_i(t) + M(t) \right)$$

mit P_i als Zellpuls und $M(t)$ als zugelassene Memory- oder Context-Projektion. Die Implementierung **MUST** eine fixed-point-kompatible, deterministische Normalisierung verwenden.

5 Kanonische Primitive

5.1 Zahl- und Hashregeln

- Alle gate-relevanten Werte **MUST** Fixed, CanonicalNumber oder rational normalisiert sein.
- Plattform-Floats **MUST NOT** in Audit-, Hash-, Gate- oder Replay-Pfaden vorkommen.
- Alle Maps **MUST** als BTreeMap oder äquivalente geordnete Struktur serialisiert werden.
- Alle Vektoren ohne semantische Reihenfolge **MUST** vor Hashing sortiert werden.
- Alle Reports **MUST** JCS- oder projektkanonisch serialisiert werden.

5.2 Basistypen

```
pub type Hash256 = [u8; 32];
pub type StableId = String;
pub type Fixed = CanonicalNumber;

pub struct EvidenceRef {
    pub kind: String,
    pub address: Hash256,
    pub relation: String,
}
```

6 Matrix-Basistypen

Diese Typen bilden das minimale Matrix-Substrat, das der Zellschicht zugrunde liegt. Eine Implementierung **MUST** diese Typen oder äquivalente, schema-kompatible Typen bereitstellen.

6.1 PhaseMatrixNode und PhaseSubnet

```
pub struct PhaseMatrixNode {
    pub node_id: Hash256,
    pub matrix_id: Hash256,
    pub null_center_id: Hash256,
    pub local_phase: Fixed,
    pub trust_state: NodeTrustState,
    pub evidence_refs: Vec<EvidenceRef>,
}
```



```

}

pub enum NodeTrustState {
    Unknown,
    Candidate,
    Active,
    Quarantined,
    Retired,
}

pub struct PhaseSubnet {
    pub subnet_id: Hash256,
    pub matrix_id: Hash256,
    pub member_nodes: Vec<Hash256>,
    pub claim_graph_hash: Hash256,
    pub truth_state_hash: Hash256,
    pub boundary_policy_hash: Hash256,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

6.2 Claims, Truth-Maintenance und Boundary

```

pub struct MatrixClaim {
    pub claim_id: Hash256,
    pub source_ref: Hash256,
    pub claim_kind: MatrixClaimKind,
    pub payload_hash: Hash256,
    pub confidence: Fixed,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub enum MatrixClaimKind {
    Observation,
    IntentCandidate,
    ClusterTraceSummary,
    BoundaryEvent,
    HandoffCandidate,
}

pub struct TruthMaintenanceReport {
    pub report_id: Hash256,
    pub checked_claims: Vec<Hash256>,
    pub accepted_claims: Vec<Hash256>,
    pub rejected_claims: Vec<Hash256>,
    pub conflict_score: Fixed,
    pub passed: bool,
}

pub struct MatrixBoundaryReport {
    pub report_id: Hash256,
    pub subject_id: Hash256,
}

```

```

pub boundary_policy_hash: Hash256,
pub violations: Vec<String>,
pub passed: bool,
}

```

6.3 MatrixTrace und Gate-Policy

```

pub struct MatrixTrace {
    pub trace_id: Hash256,
    pub source_id: Hash256,
    pub event_hashes: Vec<Hash256>,
    pub report_hashes: Vec<Hash256>,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub struct MatrixGatePolicy {
    pub require_boundary_pass: bool,
    pub require_truth_pass: bool,
    pub require_trace_ready: bool,
    pub max_conflict_score: Fixed,
}

```

7 Run Descriptor v0.3

```

pub struct PhaseMatrixRunDescriptorV3 {
    pub run_id: StableId,
    pub matrix_id: Hash256,
    pub parent_profile_hash: Option<Hash256>,
    pub operator_versions: BTreeMap<String, String>,
    pub canonicalization_version: String,
    pub thresholds: CellSubstrateThresholds,
    pub policies: CellSubstratePolicies,
    pub feature_flags: BTreeSet<String>,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub struct CellSubstrateThresholds {
    pub min_cell_coherence: Fixed,
    pub min_cluster_coherence: Fixed,
    pub min_phase_alignment: Fixed,
    pub min_morphodynamic_potential: Fixed,
    pub max_cluster_conflict: Fixed,
    pub min_convergence_score: Fixed,
    pub min_intent_tension: Fixed,
    pub max_working_state_retention_ticks: u64,
}

pub struct CellSubstratePolicies {
    pub formation_policy: ClusterFormationPolicy,
    pub morphology_policy: MorphologyPolicy,
}

```

```

pub dissolution_policy: DissolutionPolicy,
pub trace_policy: ClusterTracePolicy,
pub intent_policy: IntentPolicy,
pub matrix_gate_policy: MatrixGatePolicy,
}

```

8 Datenmodelle: PhaseCell-Schicht

8.1 TridentVector

```

pub struct TridentVector {
  pub semantic_density: Fixed,
  pub structural_coherence: Fixed,
  pub temporal_phase: Fixed,
}

impl TridentVector {
  pub fn activation_potential(&self) -> Fixed {
    // implemented with checked fixed-point multiplication
    self.semantic_density * self.structural_coherence * self.temporal_phase
  }
}

```

8.2 PhaseCell

```

pub struct PhaseCell {
  pub cell_id: Hash256,
  pub parent_node_id: Hash256,
  pub trident: TridentVector,
  pub local_resonance: Fixed,
  pub harmonization: Fixed,
  pub spiral_memory_state: Option<Hash256>,
  pub role: PhaseCellRole,
  pub lifecycle: CellLifecycle,
  pub evidence_refs: Vec<EvidenceRef>,
}

pub enum PhaseCellRole {
  Sensor,
  Resonator,
  Router,
  Validator,
  MemoryProbe,
  BoundaryGuard,
  CandidateEmitter,
  MorphologyRegulator,
}

pub enum CellLifecycle {
  Dormant,
}

```

```

    Active,
    Clustered,
    Decaying,
    Compacted,
    Retired,
}

```

8.3 CellPool

```

pub struct CellPool {
    pub pool_id: Hash256,
    pub parent_subnet_id: Hash256,
    pub cells: Vec<PhaseCell>,
    pub pool_coherence: Fixed,
    pub pool_entropy: Fixed,
    pub active_cluster_refs: Vec<Hash256>,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

CellPool-Invariante

Ein CellPool **MUST** nur PhaseCells enthalten, deren `parent_node_id` oder delegierte Subnet-Autorisierung mit dem `parent_subnet_id` kompatibel ist. Fremde Zellen **MUST** vor Clusterbildung abgewiesen oder als ExternalEvidence markiert werden.

9 LocalResonanceProcessoren

9.1 LocalResonanceProcessor

```

pub struct LocalResonanceProcessor {
    pub processor_id: Hash256,
    pub input_projection_hash: Hash256,
    pub weight_profile_hash: Hash256,
    pub nonlinearity: ResonanceNonlinearity,
    pub output_trident: TridentVector,
    pub resonance_pulse: Fixed,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub enum ResonanceNonlinearity {
    Logistic,
    TanhApprox,
    SaturatingLinear,
    PiecewiseFixed,
}

```

9.2 ResonancePulse

```

pub struct ResonancePulse {
    pub pulse_id: Hash256,
    pub source_cell_id: Hash256,
    pub trident: TridentVector,
    pub activation_potential: Fixed,
    pub phase_bin: PhaseBin,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub enum PhaseBin {
    Continuous(Fixed),
    KPolar(u32),
    Tripolar(i8),
    Quadrupolar(i8, i8),
}

```

10 MorphodynamicField

10.1 Datenstruktur

```

pub struct MorphodynamicField {
    pub field_id: Hash256,
    pub parent_cluster_id: Option<Hash256>,
    pub phase_resonance_field: Fixed,
    pub growth_field: Fixed,
    pub alpha: Fixed,
    pub beta: Fixed,
    pub morphodynamic_potential: Fixed,
    pub endogeny_score: Fixed,
    pub exogeny_score: Fixed,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

10.2 MorphologyEvent

```

pub enum ClusterMorphologyEvent {
    Grow,
    Split,
    Fuse,
    Decay,
    Replicate,
    Stabilize,
    DissolveWorkingState,
    CompactToTrace,
}

pub struct MorphologyDecision {
    pub decision_id: Hash256,
    pub cluster_id: Hash256,
}

```

```

    pub event: ClusterMorphologyEvent,
    pub morphodynamic_potential: Fixed,
    pub gate_report_hash: Hash256,
    pub allowed: bool,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

11 ResonanceCluster

11.1 Cluster-Modell

```

pub struct ResonanceCluster {
    pub cluster_id: Hash256,
    pub parent_subnet_id: Hash256,
    pub member_cells: Vec<Hash256>,
    pub cluster_trident: TridentVector,
    pub coherence_score: Fixed,
    pub conflict_score: Fixed,
    pub morphodynamic_potential: Fixed,
    pub funnel_graph_hash: Hash256,
    pub convergence_field_hash: Hash256,
    pub lifecycle: ClusterLifecycle,
    pub trace_hash: Hash256,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub enum ClusterLifecycle {
    Proposed,
    Forming,
    Active,
    Stabilized,
    Splitting,
    Fusing,
    Decaying,
    Compacted,
    Dissolved,
    Rejected,
}

```

11.2 ClusterFormationReport

```

pub struct ClusterFormationReport {
    pub report_id: Hash256,
    pub pool_id: Hash256,
    pub proposed_member_cells: Vec<Hash256>,
    pub accepted_member_cells: Vec<Hash256>,
    pub rejected_member_cells: Vec<Hash256>,
    pub phase_alignment_score: Fixed,
    pub coherence_score: Fixed,
    pub morphodynamic_potential: Fixed,
}

```

```

pub purpose_hash: Hash256,
pub trace_ready: bool,
pub passed: bool,
pub evidence_refs: Vec<EvidenceRef>,
}

```

12 FunnelGraph

12.1 Zweck

Der FunnelGraph ist keine verdeckte Routingstruktur. Er ist ein interner, auditierbarer Traversalgraph fuer einen ResonanceCluster. Er kann räumliche, zeitliche, semantische und resonante Kanten parallel halten.

12.2 Datenstruktur

```

pub struct FunnelGraph {
    pub graph_id: Hash256,
    pub cluster_id: Hash256,
    pub nodes: Vec<FunnelNode>,
    pub spatial_edges: Vec<FunnelEdge>,
    pub temporal_edges: Vec<FunnelEdge>,
    pub semantic_edges: Vec<FunnelEdge>,
    pub resonance_edges: Vec<FunnelEdge>,
    pub acyclic: bool,
    pub trace_hash: Hash256,
}

pub struct FunnelNode {
    pub node_id: Hash256,
    pub cell_id: Option<Hash256>,
    pub node_kind: FunnelNodeKind,
    pub state_hash: Hash256,
}

pub struct FunnelEdge {
    pub edge_id: Hash256,
    pub source: Hash256,
    pub target: Hash256,
    pub weight: Fixed,
    pub edge_kind: FunnelEdgeKind,
}

pub enum FunnelNodeKind { Cell, Pulse, Memory, Boundary, Intent, Trace }
pub enum FunnelEdgeKind { Spatial, Temporal, Semantic, Resonance, Evidence }

```

FunnelGraph-Invariante

Wenn `acyclic=true`, **MUST** der Graph vor Hashing topologisch validiert werden. Zyklische Graphen **MAY** nur verwendet werden, wenn der `RunDescriptor` explizit `cyclic_funnel_allowed=true` setzt und eine Replay-sichere `CyclePolicy` deklariert.

13 ConvergenceField

13.1 Datenstruktur

```
pub struct ConvergenceField {
  pub field_id: Hash256,
  pub cluster_id: Hash256,
  pub input_pulses: Vec<Hash256>,
  pub memory_projection_hash: Option<Hash256>,
  pub convergence_score: Fixed,
  pub conflict_score: Fixed,
  pub stabilized_intent_hash: Option<Hash256>,
  pub evidence_refs: Vec<EvidenceRef>,
}
```

13.2 Konvergenzregel

Ein `ConvergenceField` ist stabil, wenn:

$$G_{conv} = 1 \iff score_{conv} \geq \theta_{conv} \wedge score_{conflict} \leq \theta_{conflict} \wedge trace_ready = 1.$$

14 TensionToIntentOperator

14.1 Zweck

Der `TensionToIntentOperator` wandelt keine Feldspannung in direkte Aktion um. Er erzeugt einen `IntentCandidate`, der durch Matrix-Gates, Claims, Truth-Maintenance und gegebenenfalls Handoff weiter validiert werden muss.

14.2 Datenmodell

```
pub struct TensionToIntentOperator {
  pub operator_id: Hash256,
  pub cluster_id: Hash256,
  pub field_tension: Fixed,
  pub convergence_score: Fixed,
  pub intent_threshold: Fixed,
  pub intent_candidate_hash: Option<Hash256>,
  pub passed: bool,
  pub evidence_refs: Vec<EvidenceRef>,
}

pub struct IntentCandidate {
```



```

pub candidate_id: Hash256,
pub source_cluster_id: Hash256,
pub purpose_hash: Hash256,
pub intent_vector_hash: Hash256,
pub confidence: Fixed,
pub claim_refs: Vec<Hash256>,
pub evidence_refs: Vec<EvidenceRef>,
}

```

15 RecursiveFeedback

15.1 Bounded Feedback

RecursiveFeedback ist eine begrenzte, replayfähige Rückkopplung fuer Zell- und Clusterzustände. Sie darf keine unkontrollierte Selbstmodifikation erzeugen.

```

pub struct RecursiveFeedbackReport {
pub report_id: Hash256,
pub target_id: Hash256,
pub previous_state_hash: Hash256,
pub trident_response_hash: Hash256,
pub learning_rate: Fixed,
pub delta_hash: Hash256,
pub new_state_hash: Hash256,
pub bounded: bool,
pub passed: bool,
pub evidence_refs: Vec<EvidenceRef>,
}

```

15.2 Feedback-Regel

$$S(t) = S(t - 1) + \eta \cdot f(T(t))$$

ist nur zulaessig, wenn:

$$||S(t) - S(t - 1)|| \leq \Delta_{max}$$

und alle Change-Set-/Conformance-Regeln der Matrix erfüllt sind.

16 Dissolution und Trace-Erhaltung

16.1 Grundsatz

Dissolution-Grundsatz

Ein Arbeitszustand **MAY** kompaktieren, verfallen oder aus dem aktiven Speicher entfernt werden. Gate-Entscheidungen, EvidenceRefs, ClusterTrace, Claim-Verweise, Lifecycle-Events und Hashes **MUST** erhalten bleiben. Die Spezifikation erlaubt keine spurlose Auflösung.

16.2 Datenmodell

```
pub enum DissolutionMode {
  DropWorkingState,
  CompactToTrace,
  PersistEvidenceOnly,
  PersistClusterSummary,
  ArchiveFullState,
}

pub struct DissolutionReport {
  pub report_id: Hash256,
  pub cluster_id: Hash256,
  pub mode: DissolutionMode,
  pub working_state_before_hash: Hash256,
  pub retained_trace_hash: Hash256,
  pub retained_evidence_hashes: Vec<Hash256>,
  pub lifecycle_event_hashes: Vec<Hash256>,
  pub passed: bool,
}
```

17 ClusterTrace

```
pub struct ClusterTrace {
  pub trace_id: Hash256,
  pub cluster_id: Hash256,
  pub formation_report_hash: Hash256,
  pub funnel_graph_hash: Hash256,
  pub convergence_field_hash: Hash256,
  pub morphology_decision_hashes: Vec<Hash256>,
  pub intent_candidate_hashes: Vec<Hash256>,
  pub dissolution_report_hash: Option<Hash256>,
  pub evidence_refs: Vec<EvidenceRef>,
}
```

18 Gates

18.1 Cluster Formation Gate

$$G_{cluster} = 1 \iff G_{phase} = 1 \wedge score_{coh} \geq \theta_{coh} \wedge H \geq \theta_H \wedge purpose_valid = 1 \wedge trace_ready = 1.$$

18.2 Morphology Gate

$$G_{morph} = 1 \iff score_{endo} \geq \theta_{endo} \wedge score_{exo} \geq \theta_{exo} \wedge lifecycle_allows(event) = 1 \wedge boundary_safe = 1.$$

18.3 Intent Gate

$$G_{intent} = 1 \iff field_tension \geq \theta_{tension} \wedge score_{conv} \geq \theta_{conv} \wedge score_{conflict} \leq \theta_{conflict} \wedge trace_ready = 1.$$

18.4 Dissolution Gate

$$G_{dissolve} = 1 \iff working_state_eligible = 1 \wedge trace_persisted = 1 \wedge evidence_persisted = 1 \wedge gate_his$$

18.5 Matrix Boundary Gate

Kein Clusterereignis darf Matrix-Invarianten verletzen:

$$G_{matrix} = G_{claim} \wedge G_{truth} \wedge G_{seam} \wedge G_{lattice} \wedge G_{governance}.$$

19 Pipeline v0.3

19.1 Referenzablauf

1. Lade PhaseMatrixRunDescriptorV3.
2. Lade PhaseSubnet, HiveNode und optional externe Candidate- oder Cognition-Reports.
3. Baue oder aktualisiere CellPool.
4. Führe LocalResonanceProcessor fuer relevante Inputs aus.
5. Berechne TridentVectors und ResonancePulses.
6. Evaluiere ClusterFormationGate.
7. Erzeuge bei Gate-Pass ResonanceCluster; sonst ClusterHoldReport.
8. Baue FunnelGraph.
9. Aktualisiere MorphodynamicField.
10. Evaluiere MorphologyGate und wende erlaubte MorphologyEvents an.
11. Aggregiere Pulse im ConvergenceField.
12. Evaluiere TensionToIntentOperator.
13. Erzeuge IntentCandidate nur bei Gate-Pass.
14. Schreibe ClusterTrace.
15. Kompaktiere Working State gemaess DissolutionPolicy.
16. Übergib validierte IntentCandidates als MatrixClaims oder Handoff-Kandidaten.

19.2 Pseudocode

```
pub fn run_cell_substrate_cycle(
  rd: PhaseMatrixRunDescriptorV3,
  input: CellSubstrateInput,
) -> CellSubstrateOutcome {
  let pool = build_or_update_cell_pool(&rd, &input);
  let pulses = compute_resonance_pulses(&rd, &pool, &input);
  let formation = evaluate_cluster_formation(&rd, &pool, &pulses);

  if !formation.passed {
    return CellSubstrateOutcome::Hold(make_cluster_hold(formation));
  }

  let mut cluster = form_resonance_cluster(&rd, &pool, &formation);
  let funnel = build_funnel_graph(&rd, &cluster, &pulses);
  let morph = compute_morphodynamic_field(&rd, &cluster, &funnel);
```

```

let morph_decision = evaluate_morphology_gate(&rd, &cluster, &morph);

cluster = apply_allowed_morphology_event(&rd, cluster, morph_decision);

let convergence = update_convergence_field(&rd, &cluster, &pulses);
let intent = evaluate_tension_to_intent(&rd, &cluster, &convergence);
let trace = write_cluster_trace(&rd, &cluster, &funnel, &convergence, &intent
);
let dissolution = maybe_compact_working_state(&rd, &cluster, &trace);

CellSubstrateOutcome::Completed(CellSubstrateCycleReport {
    cluster,
    funnel,
    convergence,
    intent,
    trace,
    dissolution,
})
}

```

20 Integration mit PhaseMatrix und optionalen Handoff-Schichten

20.1 Matrix-Integration

Ein IntentCandidate **MAY** in einen MatrixClaim übersetzt werden, wenn:

- der ClusterTrace vollständig ist;
- der IntentGate bestanden ist;
- keine Truth-Maintenance-Verletzung vorliegt;
- MatrixBoundaryGate bestanden ist;
- ClaimEvidence vollständig content-addressed ist.

20.2 Handoff

Ein IntentCandidate **MAY** als Handoff-Kandidat dienen, aber **MUST NOT** direkt externe Commit- oder Finalisierungsartefakte erzeugen.

```

pub struct CellToHandoffCandidate {
    pub handoff_id: Hash256,
    pub intent_candidate_id: Hash256,
    pub cluster_trace_id: Hash256,
    pub matrix_claim_refs: Vec<Hash256>,
    pub recommended_pse_adapter: Option<String>,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

21 Outcome-Typen

```
pub enum CellSubstrateOutcome {
  Completed(CellSubstrateCycleReport),
  Hold(ClusterHoldReport),
  Rejected(ClusterRejectionReport),
  Compacted(DissolutionReport),
  MatrixBoundaryViolation(MatrixBoundaryReport),
  DeterminismViolation(DeterminismReport),
}

pub struct CellSubstrateCycleReport {
  pub report_id: Hash256,
  pub rd_hash: Hash256,
  pub pool_hash: Hash256,
  pub cluster: ResonanceCluster,
  pub funnel: FunnelGraph,
  pub convergence: ConvergenceField,
  pub intent: Option<IntentCandidate>,
  pub trace: ClusterTrace,
  pub dissolution: Option<DissolutionReport>,
}
```

22 CLI-Schnittstelle

22.1 Subcommands

```
phase-matrix cell-pool <subnet.json> --rd rd.json --out pool.json
phase-matrix cell-resonate <pool.json> --input input.json --rd rd.json --out
  pulses.json
phase-matrix cluster-form <pool.json> --pulses pulses.json --rd rd.json --out
  cluster.json
phase-matrix funnel-build <cluster.json> --rd rd.json --out funnel.json
phase-matrix morph-step <cluster.json> --funnel funnel.json --rd rd.json --out
  morph.json
phase-matrix converge <cluster.json> --pulses pulses.json --rd rd.json --out
  convergence.json
phase-matrix intent <convergence.json> --cluster cluster.json --rd rd.json --out
  intent.json
phase-matrix cluster-dissolve <cluster.json> --trace trace.json --rd rd.json --
  out dissolution.json
phase-matrix cluster-cycle <subnet.json> --input input.json --rd rd.json --out
  cycle.json
phase-matrix cluster-replay <cycle.json>
phase-matrix cluster-verify <trace.json>
```

22.2 CLI-Regeln

- **cluster-cycle** **MUST** die gesamte Pipeline ausführen und ein content-addressed Report erzeugen.

- `cluster-dissolve` **MUST** Trace-Erhaltung prüfen und bei fehlender Trace-Erhaltung fehlschlagen.
- `cluster-replay` **MUST** byte-identische Reproduktion aller IDs und Reports prüfen.
- `cluster-verify` **MUST** Formation, FunnelGraph, Convergence, Intent und Dissolution referentiell prüfen.

23 Feature Flags und Modulstruktur

23.1 Crate-Struktur

```
crates/phase-matrix/src/cell/
mod.rs
primitives.rs
trident_vector.rs
phase_cell.rs
cell_pool.rs
local_resonance_processor.rs
resonance_pulse.rs
morphodynamic_field.rs
resonance_cluster.rs
funnel_graph.rs
convergence_field.rs
tension_to_intent.rs
ouroboros_feedback.rs
dissolution.rs
cluster_trace.rs
cluster_gate.rs
handoff.rs
report.rs
replay.rs
pipeline.rs
```

23.2 Feature Flags

Feature	Bedeutung
<code>cell-substrate</code>	Aktiviert Kern der morphodynamischen Zellschicht.
<code>cell-cli</code>	Baut CLI-Erweiterungen.
<code>cell-funnel-graph</code>	Aktiviert FunnelGraph-Konstruktion und Validierung.
<code>cell-morphodynamics</code>	Aktiviert MorphodynamicField und MorphologyEvents.
<code>cell-convergence</code>	Aktiviert ConvergenceField und IntentOperator.
<code>cell-handoff</code>	Aktiviert Übergabe an Handoff-Adapter; erzeugt keinen Commit.

24 Tests

24.1 Unit Tests

1. trident_activation_is_deterministic
2. phase_cell_hash_is_stable
3. cell_pool_sorts_cells_before_hashing
4. resonance_processor_rejects_nonfinite_values
5. cluster_forms_only_above_thresholds
6. cluster_gate_fails_closed_without_trace
7. morphodynamic_potential_matches_formula
8. morphology_gate_blocks_boundary_violation
9. funnel_graph_acyclic_validation_passes
10. funnel_graph_cycle_rejected_by_default
11. convergence_field_requires_low_conflict
12. intent_gate_requires_tension_and_convergence
13. dissolution_preserves_trace
14. dissolution_preserves_evidence
15. cluster_trace_is_content_addressed
16. handoff_does_not_create_external_commit_artifacts

24.2 Integration Tests

1. cell_cycle_produces_report_or_hold
2. cell_cycle_replay_byte_identical
3. cluster_dissolve_keeps_trace_after_compaction
4. funnel_to_convergence_to_intent_happy_path
5. matrix_boundary_violation_blocks_intent_claim
6. handoff_candidate_contains_cluster_trace
7. cli_cluster_cycle_writes_report
8. cli_cluster_replay_ok
9. cli_cluster_verify_detects_modified_trace

24.3 Negative Tests

1. Core-API darf keine domänenspezifischen Adaptertypen exportieren.
2. Working-State-Kompaktion darf Trace oder Evidence niemals entfernen.
3. Cluster darf MatrixBoundaryGate nicht umgehen.
4. Cluster darf keinen externen Commit oder externes Finalisierungsartefakt erzeugen.
5. Nichtdeterministische Randomness darf nicht in Audit-, Gate- oder Hash-Pfade gelangen.

25 Definition of Done

Eine Implementierung von PHASEMATRIX-HIVEMIND-03 ist abgeschlossen, wenn:

1. `cargo test -workspace` besteht.
2. Alle neuen öffentlichen Typen dokumentiert sind.
3. Der Zellkern ohne Netzwerkabhängigkeit kompiliert.
4. Public API, CLI und Report-Schema ausschließlich domänenneutrale Core-Typen verwenden.
5. Alle gate-relevanten Werte fixed-point oder kanonisch rational sind.
6. Alle Reports content-addressed und replayfähig sind.
7. Clusterbildung fail-closed ist.
8. FunnelGraph topologisch validiert wird.
9. MorphologyEvents nur durch MorphologyGate erlaubt werden.
10. IntentCandidates nur durch IntentGate entstehen.
11. Dissolution Trace, Evidence und Gate-Historie erhält.
12. Handoff keine externen Commit- oder Finalisierungsartefakte erzeugt.
13. CLI `cluster-cycle`, `cluster-replay` und `cluster-verify` end-to-end funktionieren.
14. Golden Fixtures fuer Happy Path, Hold, BoundaryViolation und Dissolution existieren.

26 Coding-Agent-Auftrag

PATCH-ID: PHASEMATRIX-HIVEMIND-03

TITLE: Morphodynamic Resonance Cell Substrate

GOAL:

Implementiere ein eigenständiges morphodynamisches Resonanzzell-Substrat fuer `phase-matrix`. Die Implementierung MUSS alle Matrix-Basistypen und die vollständige Zell-/Cluster-Pipeline dieser Spezifikation enthalten: `PhaseCells`, `TridentVector`, `LocalResonanceProcessor`, `ResonanceCluster`, `MorphodynamicField`, `FunnelGraph`, `ConvergenceField`, `TensionToIntent`, `RecursiveFeedback`, `Dissolution`, `ClusterTrace`, `MatrixBoundaryGate` und optionalen `Handoff`.

MUST:

- Module unter `crates/phase-matrix/src/cell/` anlegen;
- Matrix-Basistypen aus dieser Spezifikation bereitstellen oder schema-kompatibel anbinden;
- alle Typen dieser Spezifikation implementieren;
- keine `platform-floats` im `Audit-/Gate-/Hash-Pfad` verwenden;
- `BTreeMap/BTreeSet` und sortierte Vektoren vor Hashing nutzen;
- `ClusterFormationGate`, `MorphologyGate`, `IntentGate`, `DissolutionGate` und `MatrixBoundaryGate` fail-closed implementieren;
- Working State Compaction erlauben, aber `Trace/Evidence/Gate-History` erhalten;
- CLI-Subcommands `cell-pool`, `cell-resonate`, `cluster-form`, `funnel-build`, `morph-step`, `converge`, `intent`, `cluster-dissolve`, `cluster-cycle`, `cluster-replay` und `cluster-verify` implementieren;
- Golden Fixtures und Tests gemaess Testplan liefern;
- Handoff nur als Kandidat erzeugen, ohne externe Commit- oder Finalisierungsartefakte.

MUST NOT:

- Trace oder Evidence durch Dissolution entfernen;

- MatrixBoundaryGate umgehen;
- externe Commit-Artefakte erzeugen;
- nichtdeterministische Randomness im Audit-Pfad nutzen;
- domänenspezifische Adaptersemantik in Core-Reports oder Core-APIs erzwingen.

SHOULD:

- Feature Flags cell-substrate, cell-cli, cell-funnel-graph, cell-morphodynamics, cell-convergence und cell-handoff bereitstellen;
- README mit Quickstart, Datenfluss, Gate-Regeln, Replay und Minimalbeispielen aktualisieren;
- Golden Fixtures fuer Happy Path, Hold, BoundaryViolation und Dissolution liefern.

27 Schlussformel

$$CellSubstrate(s) = IntentCandidate$$

nur wenn $G_{cluster} = 1 \wedge G_{morph} = 1 \wedge G_{conv} = 1 \wedge G_{intent} = 1 \wedge G_{matrix} = 1 \wedge TraceReady = 1$.

Andernfalls gilt:

$$CellSubstrate(s) = Hold(s, diagnostics).$$

Endinvariante

Die PhaseMatrix darf emergente Zellcluster erzeugen, transformieren, fusionieren, teilen und kompaktieren. Sie darf jedoch niemals ihre eigene Nachvollziehbarkeit verlieren. Emergenz ist erlaubt; nicht-auditierbare Emergenz ist verboten.
